

Hospital Management System

Object-Oriented Programming — Project Report

C++ | Single-File Implementation | main.cpp

1. Introduction

This project is a console-based Hospital Management System developed in C++ as part of the Object-Oriented Programming course. The system allows users to manage patients, doctors, appointments, and billing through a simple menu interface.

The entire application is contained in a single file (`main.cpp`) to keep it easy to compile and read. The program demonstrates the four core OOP concepts: encapsulation, inheritance, polymorphism, and abstraction.

2. Objectives

- Build a working hospital management system using C++ and OOP principles
- Apply inheritance by creating `Patient` and `Doctor` classes that share a common `Person` base class
- Use polymorphism through virtual functions so the correct `displayInfo()` method is called based on the actual object type
- Demonstrate encapsulation by keeping class data private and exposing it through getters and setters
- Keep the implementation simple, readable, and contained in one file

3. OOP Concepts

3.1 Encapsulation

All class data members are declared `private`. Public getter and setter methods are provided to access or modify the data. This protects the internal state of each object.

For example, in the `Patient` class, `patientID` and `disease` are private. They can only be read through `getPatientID()` and `getDisease()`.

3.2 Inheritance

`Person` is the base class. It holds attributes common to both patients and doctors: name, age, gender, and phone number. `Patient` and `Doctor` both inherit from `Person` and add their own specific attributes, avoiding code repetition.

Class	Inherits From	Additional Attributes
Person	(base class)	name, age, gender, phone
Patient	Person	patientID, disease
Doctor	Person	doctorID, specialization, fee

3.3 Polymorphism

`displayInfo()` is declared as `virtual` in `Person`. Both `Patient` and `Doctor` override this method with their own version.

When a `Person*` pointer holds a `Patient` or `Doctor` object and `displayInfo()` is called, the correct overridden version runs at runtime based on the actual object type. This is **runtime polymorphism**. The program includes a dedicated menu option that demonstrates this directly.

3.4 Abstraction

Complex operations like adding a patient, booking an appointment, or generating a bill are hidden behind simple function calls and menu options. The user only sees inputs and outputs — internal data storage and logic are not exposed.

4. Class Design

Person — Base Class

Stores name, age, gender, and phone. Contains a `virtual displayInfo()` method that derived classes override.

Patient — Derived from Person

Adds a unique patient ID and the patient's disease. Overrides `displayInfo()` to show patient-specific details along with the base class info.

Doctor — Derived from Person

Adds a unique doctor ID, specialization, and consultation fee. Overrides `displayInfo()` to show doctor-specific details.

Appointment

Stores an appointment ID, patient ID, doctor ID, date, and time slot. Links a patient to a doctor for a specific visit.

Bill

Stores consultation fee, medicine cost, and room charges. The total is automatically calculated when the object is created by summing all three values.

5. Features

Feature	Description
Add Patient	Register a new patient with name, age, gender, phone, and disease
View All Patients	List all registered patients with their details
Search Patient	Find a specific patient by their ID
Add Doctor	Register a doctor with specialization and consultation fee
View All Doctors	List all registered doctors with their details
Book Appointment	Book a patient with a doctor for a given date and time
View Appointments	Show all appointments with patient and doctor names

Generate Bill	Create a bill; consultation fee auto-filled from appointment if found
View Patient Bills	Show all bills associated with a specific patient
Show Polymorphism	Call <code>displayInfo()</code> via <code>Person*</code> pointer on both Patient and Doctor

6. How to Compile and Run

The project is a single file. To compile:

```
g++ main.cpp -o hospital
```

To run on Linux/macOS:

```
./hospital
```

To run on Windows:

```
hospital.exe
```

No external libraries are needed. The program runs entirely in the terminal.

Example Session

```
Main Menu
1. Patient Management
2. Doctor Management
3. Appointment System
4. Billing System
5. Show Polymorphism
6. Exit
Enter choice: 1

Patient Management
1. Add Patient
2. View All Patients
3. Search Patient
4. Back
Enter choice: 1

Enter name: Ali Khan
Enter age: 30
Enter gender: Male
Enter phone: 0300-1234567
Enter disease: Fever
Patient added. ID: 1
```

| 7. Limitations

- Data is not saved to a file — everything resets when the program exits
- Maximum of 50 records per category (patients, doctors, appointments, bills)
- No strict input format validation (e.g., date format is not enforced)

| 8. Conclusion

This project applies the four core OOP concepts through a practical example.

Inheritance avoids repetition by placing shared attributes in a base class.

Polymorphism allows different classes to define their own behavior for the same method. Encapsulation keeps data safe and accessible only through controlled methods. Abstraction hides complexity behind a clean menu interface.

The program is intentionally kept simple — one file, straightforward menus, and no unnecessary complexity — making it easy to understand, compile, and extend.